

exp201 — order-marginalized supervision for contacts-v1

contacts-v1 serialises two unordered **sets** — the sequence statements and the selected contacts — in a uniformly random order. One-hot next-token supervision therefore spends most of its budget asking the model to predict which permutation the generator happened to draw: information the model can never have, and that we do not want it to learn.

This experiment replaces the one-hot target with the **exact conditional marginal** implied by that generation process, and asks whether contacts-v1 trains materially faster on it.

Issue: #201. Phase 0 (offline accounting) is complete; Phases 1-4 are not run.

The exact targets

Writing the structure section as ``<contact> X_k Y_k``, with R_k the contacts not yet emitted at slot k :

- statement head -> uniform over the not-yet-emitted statement heads - first endpoint -> $\deg_R(p) / (2 |R_k|)$ - second endpoint -> uniform over X_k 's remaining partners - everything else -> one-hot (amino acids, markers, `<contact>` vs `<end>`)

They are a ****pure function of the token stream****, so the training A/B needs no corpus change, no tokenizer change and no data-pipeline change — it runs against the existing exp53 cache, bit-identical data on both arms.

Phase 0 result: 77% of the loss is nuisance

Measured over the whole exp53 validation split (41,954 documents, 47.8 M tokens). The "floor" is the cross-entropy an oracle that knew the structure exactly would still pay.

- nuisance floor: 2.0889 nats/token = **77.0%** of the 2.7112 val loss - informative remainder: 0.6223 nats/token

So the #117 -> #166 gap (2.7112 -> 2.6642) is a **7.6% relative** improvement in the learnable part, not the 1.7% the raw numbers suggest.

The surprise: it is mostly the sequence section

The largest single component is not the contact list. The **sequence-statement shuffle** is 1.1265 nats/token — 42% of the entire training loss — and those slots are prompt, not prediction.

That component needs no soft-target kernel to remove: the statement-head slots are identifiable from token ids alone, so zeroing them in `loss_weight`` is a ~20-line override that reuses levanter's existing fused CE. It became **Phase 1b**, a cheap arm that a soft-target win now has to beat.

And it gets worse with length

The nuisance share rises monotonically with chain length — 54% below 100 residues, 91% above 700 — because the permutation entropy grows like $\log(N!)$ while the token count grows like N . Any gain from removing it should therefore be largest exactly where the model is currently weakest (#142).

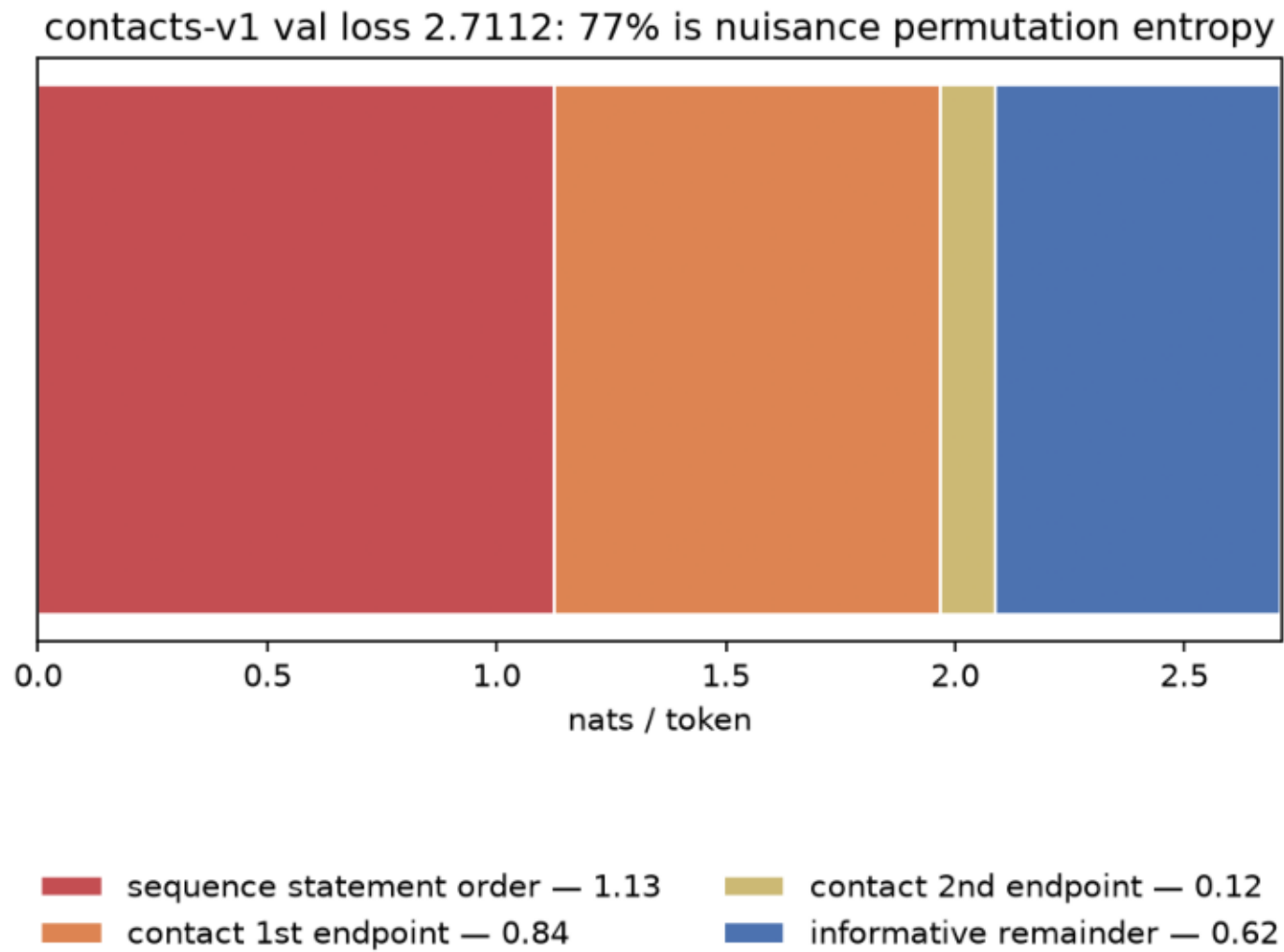
A correction the tests forced

The Monte-Carlo test contradicted the framing this experiment was written with. The one-hot target is a *sample* from the soft target, so $E[\text{hard CE}] = E[\text{soft CE}]$ exactly (verified to $1e-5$). Both share the floor $H(q)$. The soft target is a lower-variance *estimator* of the same objective, not a smaller loss number.

Consequence: swapping hard CE for soft CE cannot re-rank checkpoints on a val split this size. Phase 1 was rewritten to test the *per-slot-kind decomposition* instead. The variance/efficiency bet in Phases 1b-3 is untouched and is now the only first-order claim.

loss_budget.png

The #117 val loss decomposed. Everything left of the blue bar is the entropy of the generator's two shuffles: an oracle that knew the structure exactly would still pay it.



nuisance_vs_length.png

Nuisance floor per document vs chain length, as a % of the 2.7112 corpus-average val loss. Rises because $\log(N!)$ outpaces N : worst exactly where the model is weakest (#142).

